

INFORME ESTRATÉGICO DE PRODUCTO

Geode

Editor de notas git-native · Modelo comercial, producto y go-to-market

---

Propuesta de valor, modelo de monetización open-core, features free vs pro, modos normal y avanzado, riesgos a gestionar y plan de lanzamiento.

Preparado para MDevel Labs · Julio 2026

1. Resumen ejecutivo

Geode es un editor de Markdown local-first que usa **git como memoria automática e invisible**. Su diferencial no es el editor ni git en sí, sino la **edición no-destructiva**: el usuario nunca puede perder nada y puede experimentar sin miedo, sin necesidad de saber que hay git por debajo.

El plan es publicar el cliente **gratis y open source** para generar tracción, y monetizar los servicios gestionados (IA, sincronización sin fricción, RAG en la nube y funciones de equipo) bajo un modelo **open-core**. El mercado de entrada (wedge) es el usuario técnico de gestión de conocimiento personal (PKM), que hoy sufre el plugin de Git de Obsidian y es alcanzable por canales de bajo costo.

> **LA TESIS EN UNA LÍNEA**
> El código gratis es el canal de distribución y la confianza, no el producto. Se cobra por la conveniencia gestionada —tiempo, escala y cumplimiento— que el código abierto no resuelve por sí solo.

2. Propuesta de valor

Para el usuario, Geode no vende "git". Vende **dos promesas**:

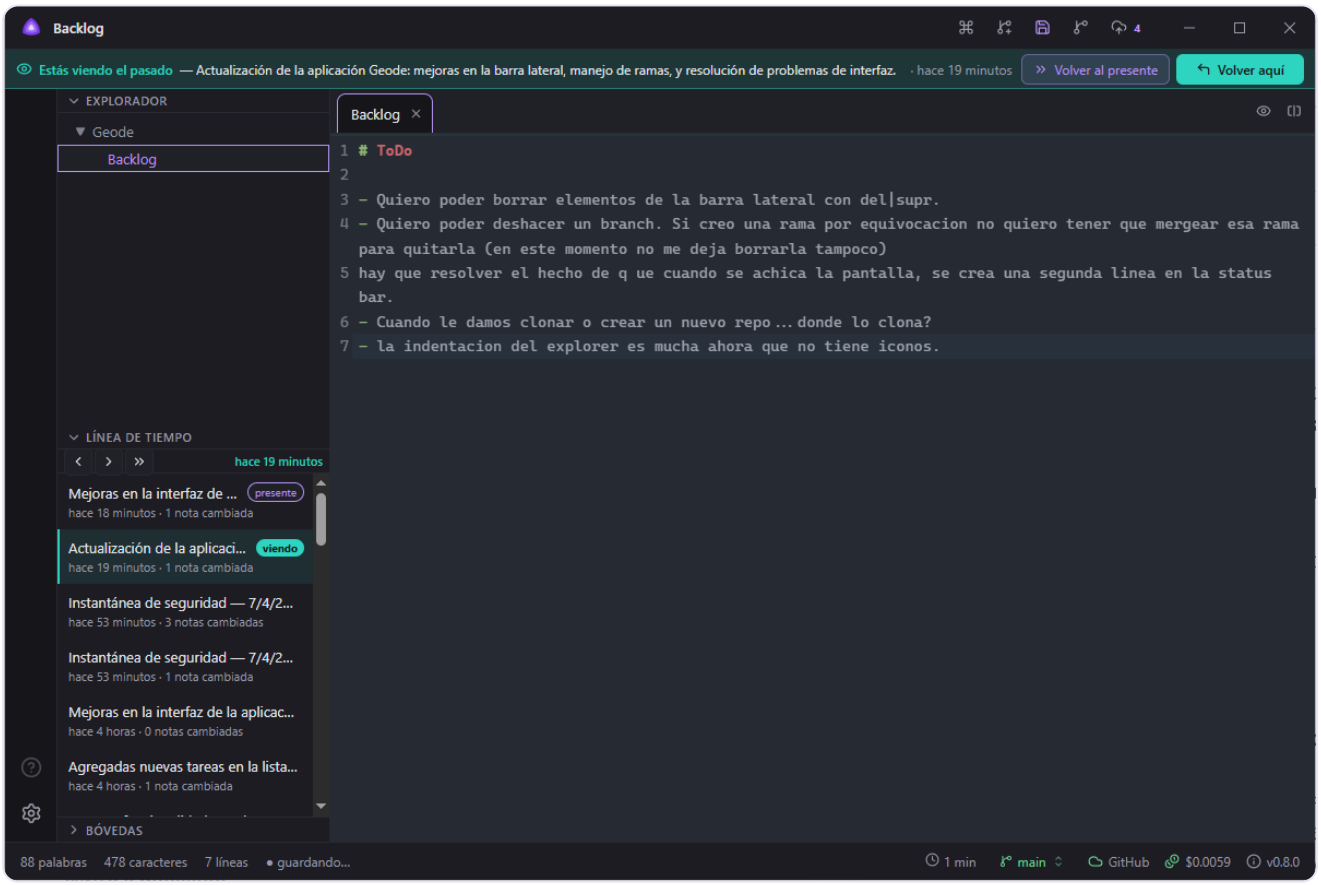
1 · Nunca perdés nada

Cada guardado queda versionado con un mensaje redactado por IA. Podés volver a cualquier punto anterior con un deslizador, en lenguaje humano.

2 · Experimentás sin miedo

Probás cambios sabiendo que el estado anterior siempre queda accesible. La edición es no-destructiva por diseño.

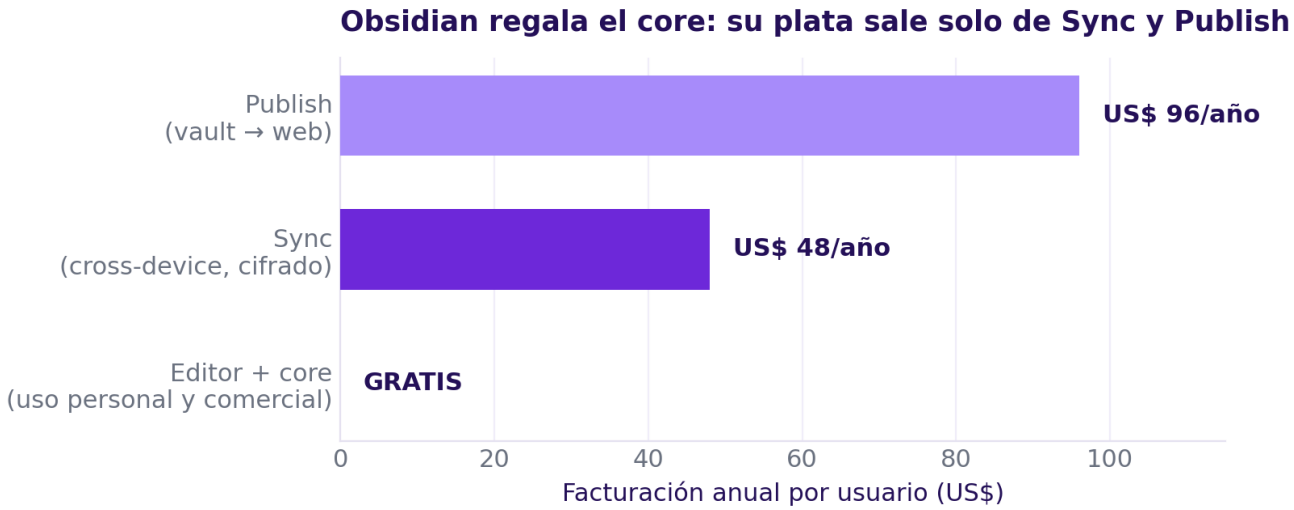
Git es el motor, no el producto —igual que una heladera tiene un compresor que el usuario nunca ve. Toda la superficie por defecto está diseñada para que esas dos promesas se cumplan **sin exponer un solo término técnico**.



Modo normal (Capa 0): línea de tiempo lineal en lenguaje humano — «Estás viendo el pasado», «Volver al presente», «hace 19 minutos · 1 nota cambiada». Git queda invisible.

3. Por qué podemos competir con Obsidian

Obsidian regala todo el core —desde 2026 incluso el uso comercial es gratis— y prácticamente toda su facturación sale de dos add-ons: **Sync** (~US$4-5/mes, sincronización cifrada entre dispositivos) y **Publish** (~US$8-10/mes, publicar el vault como sitio web). No cobran por el editor: cobran por la conveniencia de sincronizar sin configurar nada.



Fuente: página de precios de Obsidian (2026). El editor es gratis; la monetización es 100% add-ons de conveniencia.

Ahí está nuestra oportunidad, y también la trampa. El "workaround gratis" que los usuarios técnicos de Obsidian usan para no pagar Sync es **exactamente conectar el vault a git**. Geode es ese workaround, pero productizado y hecho invisible. Frente a Obsidian tenemos tres ángulos:

- **Git sin fricción y visual.** Rewind con deslizador, diff y ramas sin tocar la consola —lo que en Obsidian requiere instalar un plugin y aprender git.
- **AI-native.** Los mensajes de versión por IA y el RAG sobre el historial son parte del núcleo, no un agregado.
- **Edición no-destructiva como identidad.** "Nunca perdés nada" es un mensaje más fuerte y simple que "second brain".

> **EL REVERSO DE LA MONEDA**
> Como nuestro diferencial es la sincronización vía git —justo lo que Obsidian cobra— hay que decidir con cuidado qué se regala y qué se cobra (ver §5 y §9). Regalar el editor + git básico está bien; regalar la IA gestionada y el sync sin fricción sería quedarse sin qué vender.

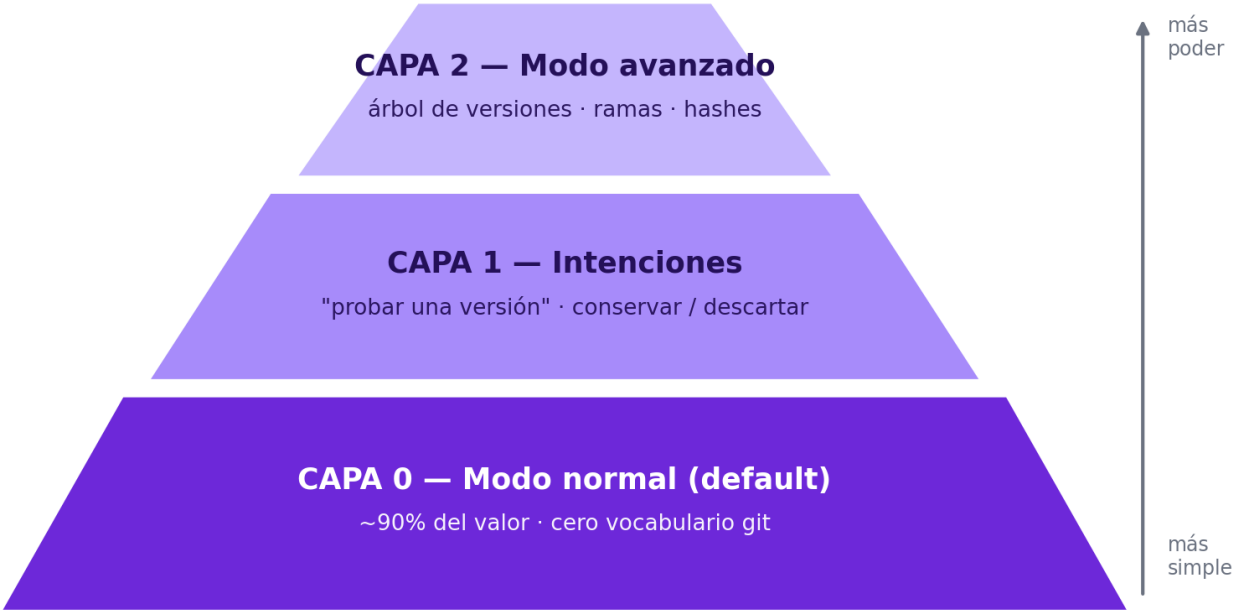
4. El producto: modo normal y modo avanzado

El principio de diseño es **mapear a intenciones, no a operaciones**. Ningún botón completa la frase "git ___"; completa la frase "quiero ___". Nunca aparecen las palabras commit, branch, merge, push, HEAD ni un hash.

El usuario quiere…	Git por debajo	Cómo lo ve
No perder lo que escribe	autocommit	(invisible)
Volver a como estaba antes	checkout / restore	"Volver a esta versión"
Ver qué cambió	diff	"Ver qué cambió"
Probar sin arruinar lo bueno	branch	"Probar una versión nueva"
El experimento sirvió	merge	"Conservar estos cambios"
El experimento no sirvió	delete branch	"Descartar el borrador"

Esto se implementa como **disclosure progresivo en tres capas**. El default es la Capa 0; el usuario técnico opta por activar el modo avanzado en Settings.

Disclosure progresivo: el default esconde git; el técnico opta por bajar



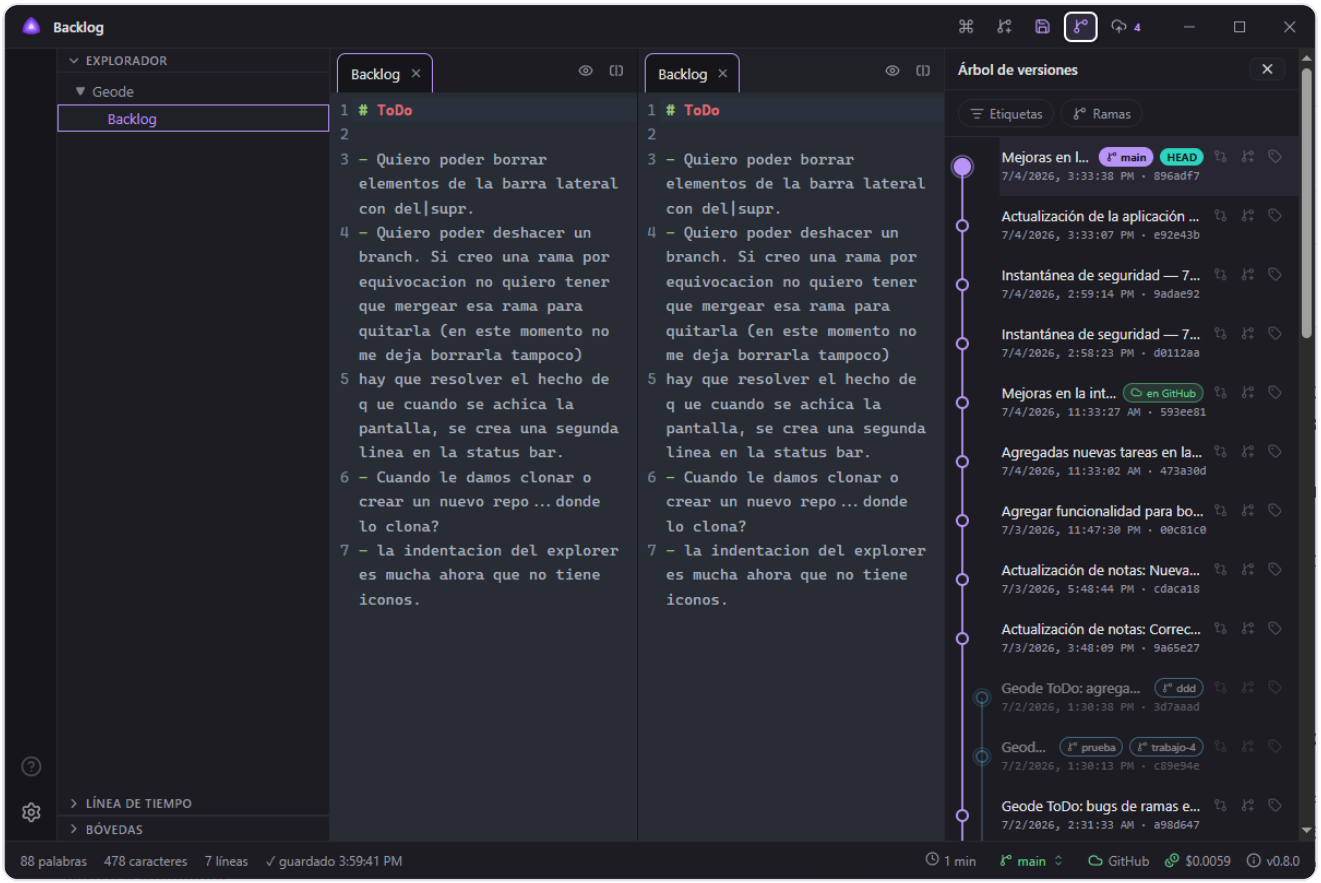
Las tres capas. El default entrega ~90% del valor sin vocabulario técnico; el DAG completo vive detrás del modo avanzado.

Modo normal (Capa 0)

Solo escribir. Autosave con historial, deslizador para volver atrás y "ver qué cambió". Timeline lineal, cero árbol, cero ramas. Con esto solo, las dos promesas ya están cumplidas.

Modo avanzado (Capa 2, opt-in)

El árbol de versiones completo, ramas desde cualquier commit, etiquetas y hashes. Es lo que atrae al usuario técnico —por eso no se elimina, se esconde.



Modo avanzado: árbol de versiones (DAG), pestaña «Ramas», etiquetas y hashes. Todo esto queda detrás del toggle de Settings.

5. Modelo comercial

Open-core: **el cliente es abierto; los servicios server-side son cerrados** (no van al repo público, para que nadie forkee el tier pago). La monetización se estructura en tres niveles.

Modelo comercial: open-core en tres niveles



El código gratis baja el costo de adquisición; una fracción convierte donde duele: tiempo, escala, compliance.

El código gratis baja el costo de adquisición. Una fracción de usuarios convierte donde el código abierto no alcanza.

La costura más limpia es la IA: el mensaje de versión por IA existe en el tier gratis **con tu propia API key (BYOK)** y en el pago **gestionado**, sin que el usuario configure nada. Misma feature, distinta conveniencia —y esa diferencia es lo que se paga.

6. Features: Free vs Pro vs Teams

Feature	Free / OSS (BYOK)	Pro / Cloud	Teams
Editor, edición no-destructiva, historial	●	●	●
Rewind, diff, "ver qué cambió"	●	●	●
Modo avanzado (árbol, ramas)	●	●	●
Mensaje de versión por IA	● key propia	● gestionado	●
Push a repo propio de GitHub	● config manual	●	●
Búsqueda semántica / RAG	● índice local	● cloud	●
Sync sin fricción (sin tocar GitHub, multi-device)	—	●	●
"Preguntale a tu vault" (RAG conversacional)	—	●	●
Publish (vault → sitio web)	—	●	●
Vaults compartidos	—	—	●
Audit trail / atribución (compliance)	—	—	●
SSO / SAML · RBAC · self-host (GitLab/Gitea)	—	—	●
SLA y soporte con contrato	—	—	●

DOS CORTES QUE NO SE NEGOCIAN

La **edición no-destructiva nunca se gatea** —es el hook de tracción, va gratis para todos. Y el **audit trail** es el mismo historial de git, revendido como "cada cambio es versionado y atribuible": argumento de compliance para el comprador B2B, sin construir nada nuevo.

7. Cómo se cobra

Free / OSS US$ 0

Cliente abierto con BYOK: el usuario pone su propia API key y su índice RAG es local. Sin costo de infra para nosotros. Es el motor de adquisición.

Pro / Cloud suscripción

Mensual/anual, en la franja de Obsidian Sync (~US$4-6/mes) como referencia de mercado. Incluye IA y sync gestionados, RAG cloud y Publish.

Teams / B2B por asiento + SLA

Precio por usuario con mínimos, más contrato de soporte. Es donde está el ticket alto: acá el comprador paga por trazabilidad y responsabilidad, no por el editor.

Supporter license one-time ~US$25

Alcancía de goodwill al estilo Obsidian Catalyst. Sirve para **validar willingness-to-pay antes de construir el Cloud**: primer experimento de monetización.

NOTA DE ARQUITECTURA — RAG

El índice vectorial es un artefacto derivado del markdown en git (se reconstruye siempre), así que no es una decisión irreversible: local en el tier free, en la nube en el pago. El pipeline es incremental por diseño —git dice qué cambió en cada commit, así que solo se re-embebe el diff. En mobile conviene BYOK-API o Cloud, no embedding on-device.

8. Caveats y riesgos a gestionar

El paywall se define ANTES de publicar, no después

El error clásico es lanzar todo gratis, entrenar a la base en que todo es gratis, y después intentar meter un paywall sobre features que la gente ya tiene. Con open source, además, alguien puede forkear y mantener gratis lo que querías cobrar. La costura (BYOK vs gestionado, cliente vs server) se arquitectura ahora, aunque el lado pago se construya en meses.

El código no es el moat

El núcleo (markdown + git + mensaje por IA) es fácil de replicar; Obsidian ya tiene plugins de git. La defensa real es el servicio hosteado, la marca y la capa RAG bien hecha —no el repositorio.

La comunidad open source cuesta tiempo de soporte

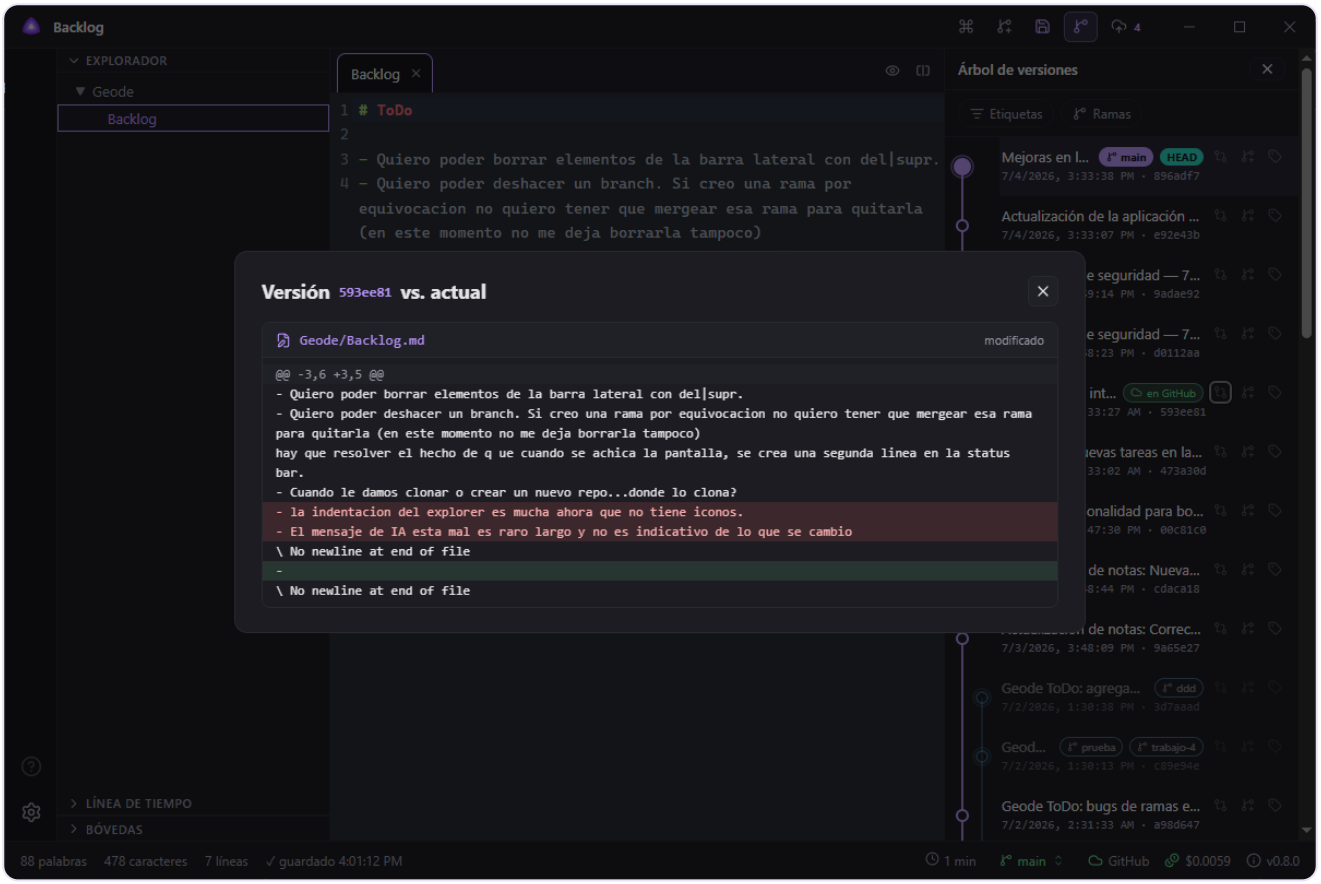
Issues, PRs, pedidos de features y soporte son un costo continuo que **compite directamente con el tiempo de construir el tier pago**. Con un portafolio ya repartido en varios productos, si Geode agarra tracción, el mantenimiento puede comerse justo las horas que se necesitan para monetizar. Conviene decidir de entrada cuánto bandwidth se le dedica.

La licencia es casi irreversible

Lo que se publica bajo MIT no se puede "cerrar" después. Si importa protegerse de que un tercero cierre y revenda el producto, evaluar AGPL. Es una decisión de Fase 0.

Fugas técnicas que rompen la Capa 0

El diff actual todavía filtra vocabulario de git —hash de versión, encabezados `@@ -3,6 +3,5 @@` y `\ No newline at end of file`. Para el usuario no técnico hay que ocultarlos por completo.



Diff actual: el hash «593ee81», los encabezados de hunk y el aviso de newline son fugas a tapar en el modo normal.

9. Estrategia para hacernos conocer

Publicar un ejecutable en GitHub no es tracción; es un repo en silencio. La tracción necesita un canal y un ángulo.

Wedge: el PKM técnico

El branching visual y el diff atraen al power user, no al escritor promedio. Ese es el público de lanzamiento —define el canal y el mensaje.

Canales

- Hacker News (Show HN)
- r/ObsidianMD · r/PKM · r/selfhosted
- Product Hunt
- Contenido técnico mostrando el flujo no-destructivo

Ángulo / mensaje

"Notas git-native con edición no-destructiva."

Novel y con gancho para la comunidad técnica: versionado real e invisible, IA que documenta cada cambio, y modo avanzado con el poder de git para quien lo quiera.

PRIMER EXPERIMENTO DE MONETIZACIÓN

Lanzar la supporter license (~US$25) junto al release gratuito. Si a la gente le gusta el producto y aun así nadie aporta, es señal tempranísima —y barata— de que el tier pago costará. Validar esto antes de invertir meses en el Cloud.

10. Pasos a seguir

Pasos a seguir

0 **FASE 0 · Decidir ANTES de publicar**
Línea free/pago cerrada · licencia (AGPL vs MIT) · qué código es abierto

1 **FASE 1 · Refactor + lanzar OSS**
Modo avanzado en Settings · Capa 0 limpia · release con BYOK

2 **FASE 2 · Tracción + validar pago**
HN / r/PKM / Product Hunt · supporter license (~US$25) para medir willingness

3 **FASE 3 · Construir el Cloud**
Sync gestionado · IA gestionada · RAG cloud · Publish

4 **FASE 4 · Teams / B2B**
Vaults compartidos · audit trail · SSO · self-host · SLA

Secuencia recomendada. La Fase 0 es la más importante y la más barata: se decide, no se construye.

Fase	Qué	Entregable
0 · Decidir	Cerrar la línea free/pago, elegir licencia (AGPL vs MIT) y definir qué código es abierto.	Matriz de features + licencia elegida (antes de publicar)
1 · Refactor + OSS	Toggle de modo avanzado en Settings, Capa 0 limpia (tapar fugas del diff), release con BYOK.	v0.x pública en GitHub
2 · Tracción	Plan de lanzamiento por canales + supporter license para medir willingness-to-pay.	Show HN / Product Hunt + primeros aportes
3 · Cloud	Construir sync gestionado, IA gestionada, RAG cloud y Publish.	Tier Pro de pago
4 · B2B	Vaults compartidos, audit trail, SSO, self-host y SLA.	Tier Teams

LA ÚNICA CORRECCIÓN AL PLAN ORIGINAL

No es "lanzo gratis y después pienso lo pago". Es "decido el corte free/pago y la licencia hoy, lanzo el lado gratis ya, y construyo el lado pago después". El orden es lo que evita el retrofit doloroso del paywall.